OpenBazaar: Signed Catalogs, Sealed Orders, and Scriptless Two-Party Escrow for a Non-Custodial Marketplace

Brian Hoffman
OpenBazaar project

November 2026

Abstract

OpenBazaar is a peer-to-peer marketplace for ordinary physical and digital goods paid in bitcoin. The original system (developed from 2014, with its last substantial releases in 2020) published stores over IPFS, carried orders over libp2p, and protected buyers with a 2-of-3 multisignature escrow whose script and moderator key appeared on chain when it was spent. This paper describes a redesign and argues which of its security and privacy properties hold, under which assumptions, and how much of each is implemented today.

Stores and listings are public, content-addressed records signed with BIP 340 Schnorr signatures over a canonical encoding. Orders are private letters sealed with HPKE to a single vendor and addressed to that vendor's onion-service mailbox; a client device talks only to its owner's node. Payments go directly to the vendor, one transaction per seller. Moderated purchases use a scriptless escrow: a MuSig2 Taproot key-path output whose timeout and dispute spends are all presigned before funding. A dispute is settled by an adaptor signature whose completing secret is split between the favored party and a moderator half that the losing party verifiably encrypts to the moderator. The moderator never holds a signing key over the funds and cannot change any payout.

We state the threat model, give the escrow protocol and its timeline precisely, tabulate what every observer learns, and label each mechanism as implemented and tested, implemented as a demonstration, specified but not implemented, or future work. At the time of writing, live Tor delivery, catalog replication, publishing costs, and integration of the two-party escrow into the node are not implemented, and the paper's analysis of those parts is a design argument rather than a description of running code.

1 Introduction

A marketplace has to do four things: let sellers publish what they offer, let buyers find it, carry an order from buyer to seller, and settle payment in a way both sides can accept. Hosted marketplaces do all four in one place. That place can see every order, hold every payment, and remove any seller. OpenBazaar's premise is that these functions can be split so that no single operator is needed for any of them, and so that the parties who must learn something (the seller who ships, the moderator who rules on a dispute) learn it only when they need to.

1.1 The original OpenBazaar

OpenBazaar began in 2014 as a prototype peer-to-peer market and was developed as `openbazaar-go` and a desktop client until late 2020 [40]. A node published its store as an IPFS directory addressed by IPNS [5]. Listings were signed by the node's identity key. Messages traveled over libp2p when

both peers were online; when the recipient was offline, the sender stored the message on its own node or an external storage host and left a pointer in the distributed hash table. Search was a separate hosted index that nodes notified on publish. The wallet supported several cryptocurrencies. Moderated purchases used a 2-of-3 multisignature script over buyer, vendor, and moderator keys, with a default timeout of 1080 hours.

That design had strengths the redesign keeps: listings were signed and an order committed to a hash of the listing it bought, so later edits could not rewrite a deal; the moderator could not take funds alone. It also had properties the redesign changes:

- A store was available only while its own node, or a pinning service, was online.
- The libp2p peer identity was long-lived and observable, and peers dialed each other directly, so the network layer linked a store identity to IP addresses.
- The default search index was a single hosted service. Delisting from it removed a store from practical discovery.
- Spending a 2-of-3 P2SH or P2WSH output reveals the full script, including the moderator's public key, so every moderated purchase was recognizable on chain and attributable to its moderator.
- A phone could not run the daemon in the background, so a vendor without an always-on desktop missed orders.
- The multi-coin wallet made wallet maintenance the dominant engineering cost.

1.2 This paper

The redesign keeps the contract between buyer, vendor, and moderator and replaces the machinery underneath. Its main ideas are:

1. **A public catalog, private orders.** Store and listing records are public, signed, and content-addressed, intended to be replicated by any node (§7). Orders are not catalog records; each is a letter sealed to one vendor and carried to that vendor's own mailbox (§9).
2. **Clients that only talk to their own node.** The user-facing app never names a peer, seed, onion address, or vendor URL. It sends requests to its owner's node, which verifies records and delivers letters (§6).
3. **Direct, per-seller payments.** A buyer pays each vendor in a separate transaction, never batching two sellers together (§10).
4. **Scriptless two-party escrow with a bounded moderator.** The escrow output is an ordinary-looking Taproot key-path output. Every non-cooperative outcome is presigned by both parties before the buyer funds it. The moderator only reveals one secret scalar that completes one of two presigned dispute outcomes (§11).

The contributions are: (i) a threat model for a marketplace with this split (§5); (ii) a precise description of the two-party escrow protocol, its message rounds, and its block-height timeline (§11); (iii) argued security properties, each with its assumptions (§13); (iv) a table of what every party and observer learns (§14); and (v) an implementation-status accounting that separates what runs from what is designed (§2, §16).

How to read the status labels. Every mechanism is marked with one of four labels. [implemented, tested] means code exists on the main branch or on a named open pull request and has automated tests that we ran. [implemented, demo] means code exists and runs, but in a configuration that is not the intended deployment (for example, fixture keys, one process playing two parties, or loopback HTTP instead of Tor). [specified, not implemented] means the behavior is designed and described here but no code implements it. [future work] means the direction is identified but not designed in detail. Where a property depends on a mechanism, the property inherits the weakest label of its dependencies. Code references are to the repository `OpenBazaar/openbazaar-rs` at main commit `b015bff`, pull request #2 (payments) at `58e539e`, pull request #3 (two-party escrow) at `dc7bc42`, and pull request #1 (desktop packaging) at `b3b81af`. Pull request #2 was squash-merged into main as `5a4e486`, with a tree identical to `58e539e`, as this paper was being finalized; we still refer to its contents as PR #2. Pull requests #1 and #3 were open and unmerged.

2 Implementation Status

Table 1 is the reference for every later claim. "Main" is commit `b015bff`; "PR #2", "PR #3", and "PR #1" are the open pull requests listed in §1.

Table 1: Implementation status of each mechanism.

Mechanism	Where	Status	What exactly exists
Catalog			
Signed store and listing records, canonical encoding, BIP 340 signatures	main	implemented, tested	Encode, decode, sign, verify; tamper tests.
Listing address = SHA-256 of body	main	implemented, tested	Order letters commit to it.
Catalog served to client	main	implemented, demo	Fixture catalog of 38 listings, verified at startup, served by the local node.
Replication by every node; replicas hold files before publish	—	specified	No replication code.
Publishing cost against spam	—	specified	No fee design in code.
Freshness and completeness of a store's catalog	—	future work	No sequence number or signed index.
Rejecting street, email, phone in public records	—	specified	Free-text fields are not filtered.
Client, node, and keys			
Client talks only to its own node on loopback, with a pairing token	main	implemented, tested	Node binds 127.0.0.1; token required for orders, addresses, and checkout.
Node rejects client-named destinations	main	implemented, tested	Any URL, onion, or peer key in a request is refused.
Desktop app starts its own node	PR #1	implemented, demo	Bundled sidecar on a loopback port, testnet4.
Phone pairs with a desktop or Android node	—	specified	The client dials only 127.0.0.1.
Addresses and buyer order records encrypted at rest	main	implemented, tested	ChaCha20-Poly1305; key file in the same directory.
BIP 39 seed with separate role paths	PR #2	implemented, tested	12 words; spend, store, escrow, scan paths; network-separated.

Mechanism	Where	Status	What exactly exists
Vendor mailbox key bound to the signed store record	—	specified	Mailbox keys come from a fixture table.
Orders			
HPKE sealing of orders and replies	main	implemented, tested	X25519, HKDF-SHA256, ChaCha20-Poly1305, base mode.
Onion mailbox storage	main	implemented, tested	Stores sealed letters without the store secret.
Arti onion client and onion-service listener	main	implemented as a library	Live dial test is ignored; not called by the node.
Delivery from buyer node to vendor node	—	specified	The demo node writes the letter to its own mailbox.
Retry from a buyer outbox when the vendor is asleep	—	specified	A closed vendor is an error today.
One sealed letter per seller in a multi-seller checkout	main	implemented, tested	Every package is validated before any letter is sealed.
Message padding	—	future work	Ciphertext length is plaintext length plus 16.
Nym for letters	—	future work	—
Payments			
One payment per seller, never batched	PR #2	implemented, tested	One recipient plus the wallet's change per transaction.
Fresh per-order vendor address	PR #2	implemented, tested	BDK, BIP 86 Taproot, per seller wallet.
Chain watcher	PR #2	implemented, tested	Esplora over clearnet; live test ignored.
Pay from the node	PR #2	implemented, tested	Mainnet needs `--network mainnet` and `"confirm": true`.
BOLT 12 offer	main	encoding only	Valid `lno1` string; no Lightning node answers it.
BIP 352 silent payment code	main	encoding only	Valid `sp1`/`tsp1` string; no scanning.
Payjoin v2	main	library only	Session types; checkout does not start one.
Chain queries over Tor	—	specified	—
Escrow			
Single-process escrow (merged)	main	implemented, demo	One process holds both secrets; fixture keys; synthetic outpoint; zero fee.
Single-process escrow on testnet4	PR #2	implemented, demo	Refund presigned before broadcast; testnet4 only; still one process.
Two-party protocol: rounds, verifiable moderator shares, refund before funding, vendor claim, moderator fee, per-order keys, self-verifying letters, signed rulings	PR #3	implemented, tested (crate only)	26 unit tests and 3 doctests. Not called by the node; no transport for the messages.
Moderator fee bond	—	future work	Noted in PR #3.
FROST moderator groups	—	future work	—
Discovery			
On-device search over headers	main	implemented	Substring filter in the client; no unit tests.
Untrusted seeds, pasted store links, remembered peers	—	specified	—
Eclipse resistance	—	specified	—

Mechanism	Where	Status	What exactly exists
PIR search	—	future work	—

3 Design Goals and Non-Goals

3.1 Goals

G1. No custody. No party other than the buyer and the vendor ever holds a key, or a set of keys, sufficient to move a buyer's payment. The moderator in particular holds no signing key over escrowed funds.

G2. Verifiable public catalog. Anyone can check that a listing was published by the store it names and has not been altered. A listing's address is derived from its content.

G3. Private orders. An order's contents, including any ship-to address, are readable only by the vendor it is addressed to. No public relay or index stores a record that a given buyer wrote to a given vendor.

G4. Bounded moderation. A moderator can choose between two payouts that buyer and vendor already signed. It cannot sign alone, redirect funds, change amounts, or learn about an order unless a party opens a dispute.

G5. Liveness without a platform. A buyer can recover escrowed funds from a vendor who disappears, and a vendor can collect from a buyer who stops responding, without anyone's cooperation beyond broadcasting a presigned transaction.

G6. Thin clients. A phone or browser client does not need to run Tor, hold spend keys, or dial the peer-to-peer network. Those jobs belong to a node the user controls.

G7. Honest degradation. When a privacy mechanism is unavailable (no Tor, no Payjoin partner, no Lightning route), the system says so rather than silently falling back.

3.2 Non-goals

- **Anonymity against a global passive adversary.** We do not claim unlinkability against an adversary that observes most of the network. Section 14 states what such an adversary learns.
- **Hiding the catalog.** Listings are public by design. A market that cannot be discovered does not serve its purpose.
- **Hiding the ship-to address from the seller or the carrier.** A seller of a physical good must learn where to send it, and the carrier must deliver it. This is a ceiling on buyer privacy for physical goods, not a gap to be engineered away.
- **Deciding factual disputes cryptographically.** Whether a parcel contained the right item is a judgment by a person. The protocol bounds what that person can do with the funds; it does not replace the judgment.
- **Reputation.** A review or rating system is outside this paper.
- **Content policy.** The protocol is designed for ordinary goods. Policies for what may be listed, and the obligations of people who operate replicas or seeds, are outside the scope of this paper.

4 System Model and Actors

Devices. A *client* is the user-facing application: an installed web app on a phone, or the same screens inside a desktop shell. A *node* is a long-running process the user controls, on a desktop computer or an Android device, that holds keys, stores records, talks to the network, and watches the Bitcoin chain. A client is paired with exactly one node. In the intended deployment the phone has no Tor client and never dials the peer-to-peer network; it talks only to its paired node and stops working when that node is unreachable [specified, not implemented]. Today the client and node run on the same machine and talk over loopback HTTP [implemented, tested].

Roles.

Buyer. Browses the catalog, sends a sealed order, and pays. For moderated purchases, funds the escrow output and holds a presigned refund.

Vendor. Publishes a signed store and listings, runs a node with an onion-service mailbox, opens letters, ships, and collects payment. For moderated purchases, holds a presigned claim.

Moderator. Named in a listing and in the signed escrow terms. Receives nothing unless a party opens a dispute. On a dispute, verifies a self-contained letter and reveals one secret scalar to the party it rules for. Paid a fixed fee by the dispute outcome.

Replica. A node that stores and serves catalog records it did not create, so that a store remains readable while its publisher is offline [specified, not implemented].

Seed. A node whose address ships with the software or is pasted by the user, used only to find other nodes. Seeds are untrusted for content: every record is verified against its signature [specified, not implemented].

External parties. The Bitcoin network and chain; a block-explorer API (Esplora) that the node queries for chain state [implemented, tested] in PR #2; Tor relays [specified, not implemented] for live use; Lightning routing nodes [specified, not implemented] for payment; a public exchange-rate endpoint the node queries for fiat estimates [implemented, tested]; and, for physical goods, a parcel carrier.

Records and messages. A *store record* names a store key, a display name, a region, and direct-payment strings. A *listing* names its store key and describes one offer. An *order letter* is an HPKE ciphertext addressed to one vendor. *Escrow messages* are the three funding rounds and the release exchange between buyer and vendor. A *dispute letter* goes from one party to the moderator, and a *ruling* comes back.

Network assumptions. We assume the Bitcoin chain makes progress and that a transaction paying a sufficient fee rate is confirmed within a bounded number of blocks; the escrow windows in §11 are sized against this assumption. We do not assume synchronous delivery between buyer and vendor nodes.

5 Threat Model

We consider the adversaries in Table 2. For each, we state what it controls and whether defending against it is in scope. "In scope" means the design claims a property against that adversary

under stated assumptions; "characterized" means we describe what the adversary achieves without claiming to prevent it.

Adversary	Capabilities	Scope
A1 Network observer	Sees, delays, and drops traffic on some links (a local network, an ISP, a single Tor relay).	In scope
A2 Malicious replica or seed	Serves, withholds, or reorders catalog records; lies about other nodes; logs who fetches what.	In scope for integrity; availability characterized
A3 Malicious counterparty	A buyer or vendor who deviates arbitrarily from the protocol, including going silent.	In scope
A4 Malicious moderator	Rules dishonestly, rules for both sides, refuses to rule, or logs disputes.	In scope for funds; ruling quality characterized
A5 Moderator colluding with one party	The moderator and either the buyer or the vendor act together.	Characterized
A6 Compromised device	Reads files or memory on a user's node or client.	Characterized
A7 Global passive adversary	Observes most network links and the chain.	Out of scope; characterized

Table 2: Adversaries considered.

A5 is inherent to any escrow with a third-party arbiter: if the arbiter and one party agree, the arbiter can rule for that party. The protocol limits A5 to choosing the presigned outcome that favors the colluding party; it cannot pay a third party or change amounts (§13). A6 is out of scope except for the at-rest encryption described in §8, whose limits we state. A7 is out of scope for anonymity, but §14 lists what it learns.

Cryptographic assumptions. Discrete logarithm hardness in the secp256k1 group, with hash functions modeled as random oracles for the Schnorr, MuSig2, and sigma-protocol arguments; collision resistance of SHA-256; IND-CCA2 security of HPKE base mode with the X25519/HKDF-SHA256/ChaCha20-Poly1305 suite [4]; and AEAD security of ChaCha20-Poly1305 [38]. Implementations come from established libraries (`libsecp256k1` through the `bitcoin` and `musig2` crates, the `hpke` crate, Arti for Tor). We assume these are correct; we did not audit them.

6 Architecture

Figure 1 shows the components. The repository is a Rust workspace with five crates and two applications:

- `crates/protocol`: store, listing, order, and dispute record types, canonical encoding, and Schnorr signing.
- `crates/tormail`: order sealing, the onion mailbox, and the Arti transport.
- `crates/pay`: BOLT 12 offers, BIP 352 codes, and Payjoin v2 session wrappers.
- `crates/escrow`: the scriptless escrow.

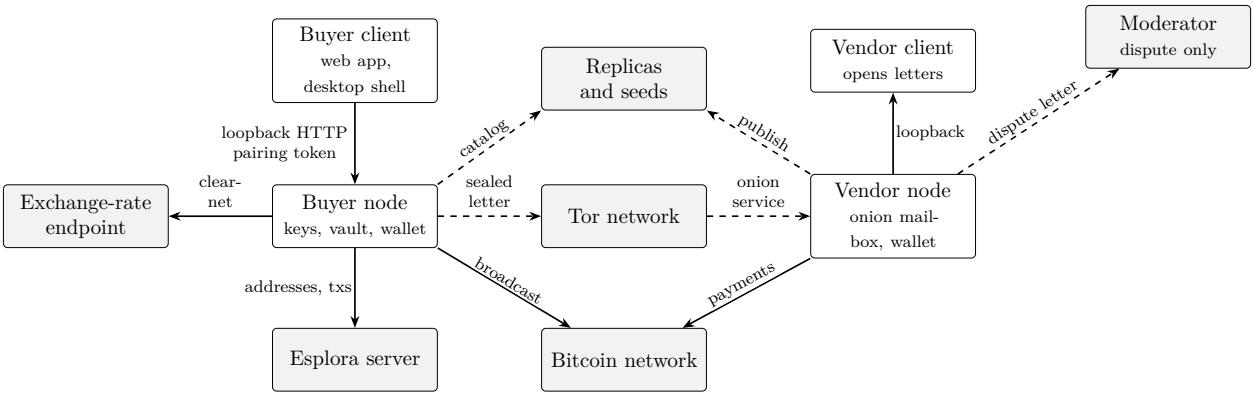


Figure 1: Components and data paths. Solid arrows exist in code on main or an open pull request; dashed arrows are specified but not implemented. In today's demonstration the buyer node and vendor node are the same process, and the sealed letter is written to that process's own mailbox directory. The phone-to-node pairing is not drawn; the client currently reaches only a node on the same machine.

- `crates/node`: the node binary (`ob-node`), its loopback HTTP server, checkout, the encrypted vault, and (in PR #2) the wallet and chain watcher.
- `apps/mobile`: the client, an Expo (React Native) application exported for the web.
- `apps/desktop`: a Tauri 2 shell that hosts the same web export and, in PR #1, starts a bundled node.

6.1 The client-node boundary

The node listens only on `127.0.0.1` [implemented, tested]. It refuses requests whose TCP peer is not a loopback address. On first start it writes a 256-bit random pairing token to a file with mode `0600` and prints it once. Requests that place orders, read or write the address book, or read the buyer's order list must carry the token in an `X-Ob-Pair` header. A loopback caller can fetch the token with `GET /pair`; a non-loopback caller cannot. Cross-origin responses name only the shop origin, the node's own origin, and (in PR #1) the desktop webview origins, so a web page from another origin cannot read the token through the browser.

This boundary protects against remote callers and against other web origins in the user's browser. It does not protect against another process running as any user on the same machine: such a process can call `GET /pair` and then act as the client. We list this in §15.

The node refuses any request in which the client names a destination [implemented, tested]: a JSON key such as `peer`, `onion`, `seed`, `replica`, or `url`, or any string value containing `://` or `.onion`. A client therefore cannot direct the node to dial an address of the client's choosing. The vendor is always the store that signed the listing the node already holds.

6.2 Intended deployment

A vendor runs a node that stays awake (the desktop app with its bundled node, or an Android device), publishes the catalog, runs an onion service that accepts sealed letters, and watches the chain. A buyer uses the same software; a buyer on an iPhone pairs the installed web app with their own desktop or Android node [specified, not implemented]. All peer-to-peer traffic, including catalog fetches, publishing, and letter delivery, is intended to run over Tor between nodes [specified, not implemented].

6.3 What runs today

On main, `ob-node serve` loads a fixture catalog, verifies every signature, and serves it on loopback. Checkout validates every requested listing against that catalog, seals one letter per seller, and writes each letter into the node's own mailbox directory: in the demonstration, buyer and vendor share one node [implemented, demo]. PR #1 bundles the node with the desktop app and starts it on a free loopback port. PR #2 adds real seeds, wallets, testnet4 payments, and a chain watcher. No code path sends anything over Tor.

7 Catalog and Replication

7.1 Records

A *store record* [implemented, tested] contains a display name, a region, two direct-payment strings (a BOLT 12 offer and a BIP 352 code), and the store's 32-byte x-only public key S, which is the store identifier. A *listing* [implemented, tested] contains S, a title, a category, a price in satoshis, a ship-from region, a description, a seller display name, a stock count, the date it was signed, whether it accepts a moderator, a kind (physical or digital), an optional shared product key, flat shipping in satoshis, and the subtotal above which shipping is waived. Neither record has a field for a legal name, an email address, a phone number, or a street address. The free-text fields are not checked for such content; rejecting it at encoding time is specified but not implemented [specified, not implemented].

7.2 Canonical encoding

Each record body is a fixed sequence of fields [implemented, tested]: a four-byte magic (`OBST` for stores, `OBLS` for listings), a one-byte version, then fields in a fixed order. Integers are little-endian of fixed width. Strings are a two-byte little-endian length followed by UTF-8 bytes; blobs are a four-byte length, capped at 8 MiB. The decoder rejects a wrong magic, a wrong version, invalid UTF-8, truncation, and trailing bytes. Because every variable-length field is length-prefixed and the field order is fixed, distinct records have distinct encodings, and a decoded record re-encodes to the same bytes (a property the tests check).

7.3 Signatures and addresses

The signature is a BIP 340 Schnorr signature [54] over a tagged hash of the body:

$$\sigma = \mathsf{Sign}_s\big(\mathsf{H}_{\mathrm{tag}}(\mathit{body})\big), \quad \mathsf{H}_{\mathrm{tag}}(m) = \mathrm{SHA256}\big(\mathrm{SHA256}(\mathrm{tag}) \,\|\, \mathrm{SHA256}(\mathrm{tag}) \,\|\, m\big),$$

with tags `ob/store/v1` and `ob/listing/v1`, so a signature over one record type is not valid for another. Signing uses BIP 340's deterministic nonce derivation without auxiliary randomness. The signature is appended to the body. A listing can only be signed by the secret key of the store it names: the signing function refuses otherwise, and verification checks the signature against the S inside the listing body.

A listing's address is SHA256(*body*), computed over the body without the signature [implemented, tested]. An order commits to this address, so a vendor who later edits a listing (for example, raising the price) produces a new address and cannot claim that the buyer agreed to the new terms. The node refuses an order for an address that is not in its verified catalog.

7.4 What the signature does not give

A valid signature proves that the store key holder signed those bytes at some time. It does not prove that the listing is current or that a set of listings is complete. A replica can serve an older, validly signed version of a listing (an old price or stock count), and it can omit listings entirely, without either being detectable by signature checks alone. Addressing a listing by content hash makes an order unambiguous about which version it bought, but does not tell a buyer whether a newer version exists. A per-store sequence number or a signed index of current listings would address freshness and completeness; neither is designed in detail [future work].

7.5 Replication

The intended design [specified, not implemented] has every node store the signed headers of the catalog it has seen, so that a store remains browsable while its publisher is offline, and requires images and files to be held by at least one replica before a listing counts as published. Because records are self-authenticating, a replica needs no trust for integrity: a client or node that receives a record from anyone checks the signature and the content hash. Replicas are trusted only for availability and see who fetches what (§14).

An open catalog invites spam. The design calls for each published header to carry a cost that receiving nodes check, such as a small bitcoin fee [specified, not implemented]; no fee mechanism is designed in detail, and none is implemented. Until one exists, replication would be limited to stores a node has chosen to follow or has received through a pasted link.

On main, the catalog is a fixture of one store and 38 listings across three sellers, signed at startup with fixture keys and verified before it is served [implemented, demo]. In the fixture, the three sellers are display names on listings under a single store key; the protocol itself assigns one store key per vendor.

8 Identity and Keys

8.1 Key roles

The design gives each user one backup phrase and derives a separate key for each role, so that a key used to sign the public catalog is never the key that spends coins. PR #2 implements this [implemented, tested] with a 12-word BIP 39 phrase [41] and BIP 32 derivation [53]:

Role	Derivation path
Spend wallet (BIP 86 Taproot [11])	`m/86'/c'/0'/{0,1}/*`
Store signing key	`m/1984'/c'/0'/0'`
Escrow key i	`m/1984'/c'/1'/i'`
BIP 352 scan key	`m/352'/c'/0'/1'/0`
BIP 352 spend key	`m/352'/c'/0'/0'/0`

The coin type c is 0 on mainnet and 1 on testnet4, and each network has its own data directory, so a phrase used on both networks never yields the same key on both. A test checks that all role keys are pairwise distinct across both networks. Escrow keys are indexed, and the index is advanced and persisted before a key is used, so each escrow gets a new key.

On main, every key is a fixture: a scalar computed by hashing a fixed label [implemented, demo]. The demonstration node holds the store key, the vendor spend and scan keys, a buyer key, and the moderator's secret key in one process. Those fixtures exist so the demonstration can run end to end on one machine; they are not a security configuration.

8.2 The mailbox key

Orders are sealed to an X25519 public key belonging to the vendor (§9). That key must be bound to the store, so that a buyer who verified a listing also knows which key to seal to. The intended binding is a field in the signed store record [specified, not implemented]. Today the store record has no such field: the node looks up the vendor's mailbox key in a fixture table keyed by seller display name [implemented, demo]. For one fixture seller, the mailbox key is the same 32-byte string as the store's secp256k1 signing key, interpreted as an X25519 scalar. Reusing one secret across two groups and two protocols has no known attack in this specific combination, but it is poor hygiene; the mailbox key should be derived on its own path [specified, not implemented].

8.3 Encryption at rest

The node stores the buyer's address book and the buyer's own order records encrypted with ChaCha20-Poly1305 under a random 256-bit key and a random 96-bit nonce per write [implemented, tested]. PR #2 stores the BIP 39 phrase and the wallet state the same way [implemented, tested]. The key is in a file named `vault.key`, mode `0600`, in the same data directory.

This protects against the encrypted files being copied or synced on their own, for example by a backup tool that skips key files, and keeps addresses out of logs and the catalog. It does not protect against anyone who can read the data directory, since the key sits beside the ciphertext. It is not a defense against a compromised device (A6). Wrapping the key with an operating-system keystore or a passphrase is not implemented.

The buyer's order record keeps the label of the address used (for example, "Home"), not the street.

9 Order Sealing and Transport

An order is a private letter to one vendor, not a catalog record and not an event on a public relay. Figure 2 shows the flow.

9.1 Order contents

The order plaintext [implemented, tested] (magic `OBOD`, version 1) contains: the address of the first listing; its quantity; an options map of up to 16 string pairs, which carries the settlement type, the shipping charge, a delivery note for digital goods, and the address and quantity of any further listings from the same seller; the ship-to text (at most 1024 bytes) for physical goods; the payment rail; and a fresh X25519 public key R that the vendor uses to reply. The order carries no long-term buyer identity key and no buyer signature. The vendor learns who the buyer is only from what the buyer writes into the letter (for a physical good, the ship-to) and from the payment.

9.2 Sealing

The node seals the plaintext with HPKE [4] in base mode with the suite DHKEM(X25519, HKDF-SHA256) [32, 33], HKDF-SHA256, and ChaCha20-Poly1305 [38] [implemented, tested]. The recipient key is the vendor's mailbox key P_V; the HPKE `info` string is `openbazaar-tormail-order-v1`; the associated data is empty. The sealed letter is

$$\texttt{OBSL} \,\|\, \texttt{0x01} \,\|\, enc \,\|\, \mathrm{len}_{32}(ct) \,\|\, ct,$$

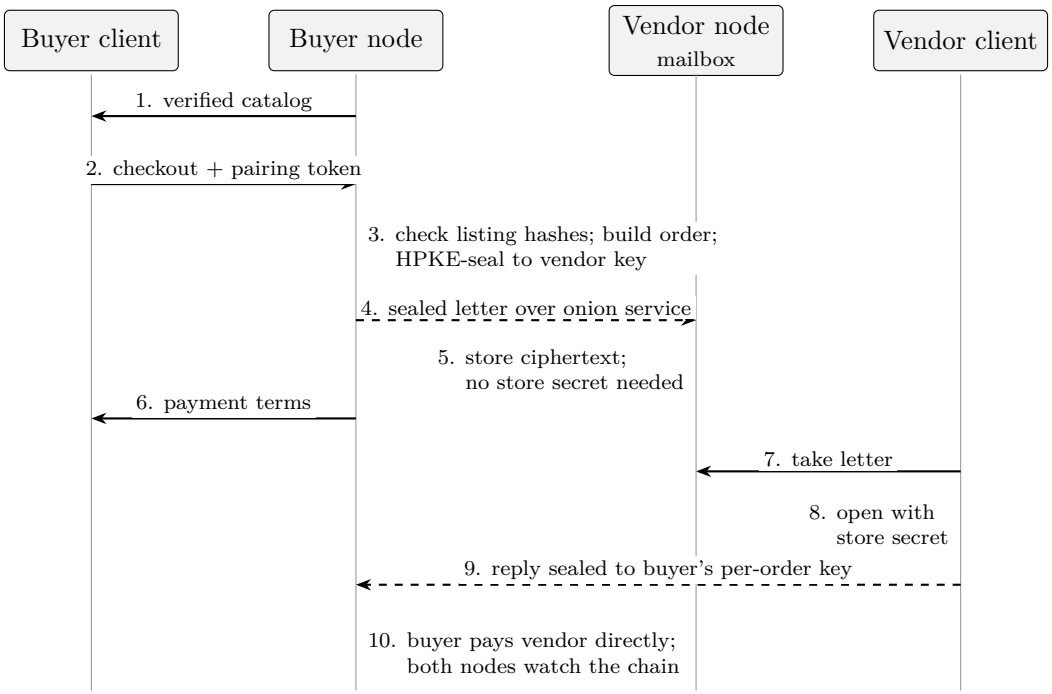


Figure 2: Order flow for one seller. Dashed messages are specified but not carried by any transport today; in the demonstration, step 4 writes the letter into the same node's mailbox directory. Steps 7–9 exist as library functions exercised by tests; the node exposes no endpoint for them yet. A multi-seller checkout repeats steps 3–10 independently for each seller.

where *enc* is HPKE's 32-byte encapsulated ephemeral key. Letters larger than 64 KiB are rejected. A vendor reply is sealed the same way to the buyer's per-order key R with `info` `openbazaar-tormail-reply-v1` and a size limit of 8 KiB. Because R is generated per order and travels inside the ciphertext, no relay or mailbox needs to remember which buyer wrote to which vendor in order to route a reply.

Base mode means letters are not authenticated to a sender: anyone who knows P_V can produce a letter that the vendor will decrypt. This is deliberate, since a buyer should not need a long-term identity to order, but it means the vendor must treat an order as an offer that becomes real when paid. It also means that a vendor's mailbox can be filled with junk letters; the mailbox caps letter size but does not rate-limit.

Letters are not padded. The ciphertext is exactly 16 bytes longer than the plaintext, so an observer who sees a letter at rest or in transit learns its length, which correlates with the length of the ship-to and the number of lines. Over Tor, a letter of a few hundred bytes fits in a small number of fixed-size cells, which hides the exact length from relays but not the approximate size. Padding to fixed buckets is future work [future work].

9.3 Multi-seller checkout

A checkout may contain packages from several sellers. The node validates every package (each listing exists in the verified catalog, each package is one seller and each seller one package, moderated packages only contain listings that accept the moderator) before sealing anything, then seals one letter per seller to that seller's key [implemented, tested]. A seller learns only the lines in its own package. Each package gets its own payment instructions and its own payment (§10).

9.4 The mailbox

The vendor's node accepts letters into a mailbox directory without loading the store secret [implemented, tested]: a letter is written to a temporary file and renamed to a random 128-bit identifier, so the identifier is not derived from the sender. The vendor's client later takes letters and opens them with the store secret. A node that is compromised while the vendor is away therefore holds ciphertext, not orders, unless the mailbox secret is also stored on it.

9.5 Transport

The `tormail` crate contains a transport over Tor using Arti [50]: `ArtiOrderTransport` dials a vendor's v3 onion address [51] on a fixed virtual port and writes one framed letter (`OBML`, version 1); `start_onion_listener` launches an onion service, accepts one stream per rendezvous, rejects streams for any other port, and writes the letter into the mailbox. Onion addresses are parsed and checksummed by Arti. This code is a library [implemented as a library]: the test that dials a live onion service is marked ignored, and no part of the node calls the transport or the listener. Tests use an in-memory transport.

The intended behavior [specified, not implemented] is that the buyer's node dials the vendor's onion service, delivers the letter, and, if the vendor's node is unreachable, keeps the letter in an outbox and retries. Today a vendor without an awake node is treated as closed. No third-party relay stores letters in either design.

Using an onion service for the vendor's mailbox means the vendor's IP address is not revealed to buyers, and the buyer's node connects through a Tor circuit, so the vendor does not learn the buyer's IP address. Tor does not protect against an adversary who observes both ends of a circuit [16, 29]; §14 discusses this. Carrying letters over a mix network such as Nym [15, 43] to resist timing correlation is future work [future work].

10 Payments

All payments are in bitcoin. There is no swap service, mixer, ecash mint, or second asset in the design or the code. Direct payment goes from the buyer to the vendor with no intermediary; moderated payment goes through the escrow of §11.

10.1 One payment per seller

A multi-seller checkout produces one order per seller, and each order is paid by its own transaction [implemented, tested] (PR #2). The pay-from-node path builds a transaction with exactly one recipient output and, if needed, one change output back to the buyer's own wallet. Two sellers are never paid in one transaction, because a transaction with outputs to two vendors would tell anyone watching the chain that the same person bought from both.

Separate transactions are necessary for unlinkability, not sufficient. If the wallet funds the second payment with the change from the first, the common-input-ownership and change heuristics [35] link the two payments. At `58e539e` coin selection skips unconfirmed coins received from others but deliberately keeps the wallet's own unconfirmed change spendable, so paying two sellers in quick succession can chain the second payment onto the first payment's change. The wallet does not currently avoid this; choosing inputs so that one checkout's payments do not share ancestry is future work [future work].

10.2 On-chain direct payment

In PR #2, each direct order receives a fresh receive address from the vendor's BDK descriptor wallet (BIP 86 Taproot) and a BIP 21 [46] URI for the exact amount [implemented, tested]. The node's chain watcher queries an Esplora server for transactions paying each watched address and marks an order as payment seen and then paid once the required confirmations (one on testnet4, three by default on mainnet) are reached. The node can also pay an order from its own wallet. The default network is testnet4 [28]. Mainnet is selected only by the exact flag `--network mainnet`, and a mainnet payment from the node additionally requires `"confirm": true` in the request.

Because the demonstration node holds both the buyer wallet and the vendor wallets, a single node can both pay and observe an order [implemented, demo]. In a deployment the buyer and vendor nodes are different machines, and each watches only its own addresses.

The Esplora server. Every chain query names a specific address to the Esplora server [7]. The defaults are `mempool.space` on mainnet and, on testnet4, `mempool.space` with `blockstream.info` as an ordered fallback; the node checks that each server's genesis block matches its network before using it. Whichever server answers learns the full set of addresses the node watches, their timing, and the IP address of the node, which together link a user's orders to each other and to a network location; once the node falls back, the second operator learns the same. On mainnet the node prints a warning when the server is public. Routing chain queries over Tor [specified, not implemented] would hide the IP address but not the address set; running one's own Esplora or Bitcoin Core node, or using compact block filters, would hide both. None of these is implemented.

10.3 Lightning: BOLT 12

A store record carries a BOLT 12 offer [45], and direct checkout returns an offer for the order's exact amount [encoding only]. The offer is a valid `lno1` string built with LDK's offer builder, signed with the store's signing public key, and carrying no blinded path. No Lightning node in the system answers invoice requests for it, so it cannot be paid today.

Two privacy points apply once it can. An offer without a blinded path names the issuer's node key, so the offer links the store to a Lightning node. Using the store signing key as the offer's issuer key, as the current code does, also links the catalog identity to that Lightning identity; a separate key is the intended design [specified, not implemented]. On a payment, every routing hop sees the amount it forwards and its neighbors; the recipient's position is hidden from the payer only to the extent that a blinded path of more than one hop is used [30].

10.4 Silent payments: BIP 352

The store record also carries a BIP 352 silent-payment code [3] [encoding only]. A silent-payment code lets a payer derive a fresh output for each payment from a static public code, so a single published code does not lead to address reuse. The code is constructed from the scan and spend public keys and parsed with checksum validation. The node does not scan the chain for payments to the code. A buyer who pays the code today creates an output the vendor's node will not detect until scanning exists; the funds are recoverable with the scan and spend keys but the order would not be marked paid. The intended design keeps the scan key, which is watch-only, on the vendor's node so that scanning does not hand the key to a public indexer [specified, not implemented].

10.5 Payjoin: BIP 77

Payjoin v2 [23] lets a receiver add an input to the buyer's transaction through an asynchronous directory reached over Oblivious HTTP [52], which breaks the assumption that all inputs belong to the payer. The `pay` crate wraps the `payjoin` library's v2 sender and receiver sessions and the step where the vendor contributes an input [library only]. No checkout starts a Payjoin session, and the payment terms carry no `pj=` endpoint. When Payjoin is used, the vendor learns which coins the buyer spent, and the buyer learns one of the vendor's coins.

10.6 What the chain shows

For an on-chain direct payment, a chain observer sees an ordinary transaction with one or two outputs. The amount is the listing price times quantity plus shipping. Prices are public, so an amount that matches a distinctive price can identify the listing that was bought, even though nothing else in the transaction refers to OpenBazaar. Round prices reduce this and do not remove it. For escrow, see §11.10.

11 Scriptless Escrow

A moderated purchase protects the buyer against non-delivery and the vendor against a buyer who will not release payment, with a moderator to decide disputes. The original OpenBazaar used a 2-of-3 multisignature script for this. The redesign keeps the economic model (a moderator together with one party can decide the outcome) and changes three things: the moderator holds no key over the funds; every outcome looks like an ordinary single-key Taproot spend; and the moderator can choose only between two payouts that buyer and vendor both signed before funding.

This section first states what is merged and what the open escrow pull request changes, then specifies the two-party protocol.

11.1 Status of the escrow code

Merged (main). The escrow crate on main computes a MuSig2 key-path funding output, pre-signs a locktime refund, and builds adaptor signatures for both dispute outcomes [implemented, demo]. It does all of this in one function, `fund`, that takes both the buyer's and the vendor's secret keys. A review of the merged code found that, in addition: the node calls it with fixture keys; the funding outpoint is synthetic; the presigned refund is not kept by the node; there is no presigned vendor default, so a buyer who stops responding cannot be overridden before the refund height; the encrypted moderator halves carry no proof that they decrypt to the right value; there is no moderator fee; network fees are zero; and the funding address is the same for every order between the same buyer and vendor keys. The merged escrow is a demonstration of the transaction shapes, not a protocol between two parties.

PR #2. The payments branch keeps the single-process ceremony but restricts it to testnet4 (mainnet refuses escrow), derives a new escrow key per order for each side, adds a 500-satoshi spend fee, and enforces the order "sign the funding transaction, run the ceremony for that exact outpoint, check and store the refund, then broadcast" [implemented, demo]. Its responses mark escrow as an unsafe demonstration.

PR #3. The two-party protocol below is implemented [implemented, tested] in the escrow crate on branch `escrow-two-party` (we ran its 26 unit tests and 3 doctests). Each party's state machine holds only that party's secret key, and the parties exchange three serializable messages. The node does not call this protocol yet, no transport carries its messages, and the old single-process function remains behind a deprecated feature so the node demonstration still builds. Everything in §11.2–§11.10 describes PR #3; we note where its behavior depends on a caller obligation that no code yet fulfills.

11.2 Notation

G generates the secp256k1 group of prime order n. Lowercase letters are scalars and uppercase letters are points. H_{tag} is a BIP 340 tagged hash.

- Long-term keys: buyer $(\bar{b}, \bar{B})$, vendor $(\bar{v}, \bar{V})$. Moderator (k, K).
- Order secret s: 32 bytes known only to buyer and vendor. Per-order keys $b = \bar{b} + \mathsf{H}_{\text{ord}}(\bar{B}, s)$ and $B = bG$, and likewise v, V.
- Aggregate key: $P = \mathsf{KeyAgg}(B, V)$ under BIP 327 [37] with the two keys sorted; output key $Q = P + \mathsf{H}_{\text{TapTweak}}(P)\,G$, the BIP 341 [55] tweak with no script tree. The funding output is P2TR with output key Q and value F.
- Terms $\tau = (B, V, K, F, f, \varphi, h_D, h_C, h_R, S_B, S_V, S_M)$: network fee f, moderator fee φ, dispute deadline h_D, vendor claim height h_C, refund height h_R, and payout scripts for buyer, vendor, and moderator. Both parties sign τ; the signed terms are τ^*.
- Outpoint o of the funding output.
- For a dispute branch favoring party $X \in \{B, V\}$, with $\bar{X}$ the other party: the *party half* t_X, $T_X = t_X G$, derived by X from its per-order secret key and the branch; the *moderator half* u_X, $U_X = u_X G$, sampled by $\bar{X}$; the adaptor point $Y_X = T_X + U_X$.
- $\mathsf{VEnc}_K(u; ctx)$: verifiable encryption of u to K, bound to context ctx (§11.9). $\pi^{\text{pok}}(t)$: Schnorr proof of knowledge of t.

11.3 Terms and transactions

Before any funding round, the parties agree on τ and each checks it under its own local policy: the three keys are distinct; $0 < h_D < h_C < h_R$ are block heights with $h_C - h_D$ and $h_R - h_C$ at least a minimum window (144 blocks each by default); the moderator fee is at least the P2TR dust limit of 330 satoshis; every payout is at least dust; and every presigned transaction meets a minimum fee rate measured on its actual signed size [implemented, tested]. The moderator key must match the key the party already trusts from the listing, so a counterparty cannot substitute a different moderator [implemented, tested].

From τ and o, each party derives the same five spends (Figure 3). Each has one input spending o with sequence `0xfffffffe` (so `nLockTime` is enforced) and is signed as a BIP 341 key-path spend with `SIGHASH_DEFAULT`:

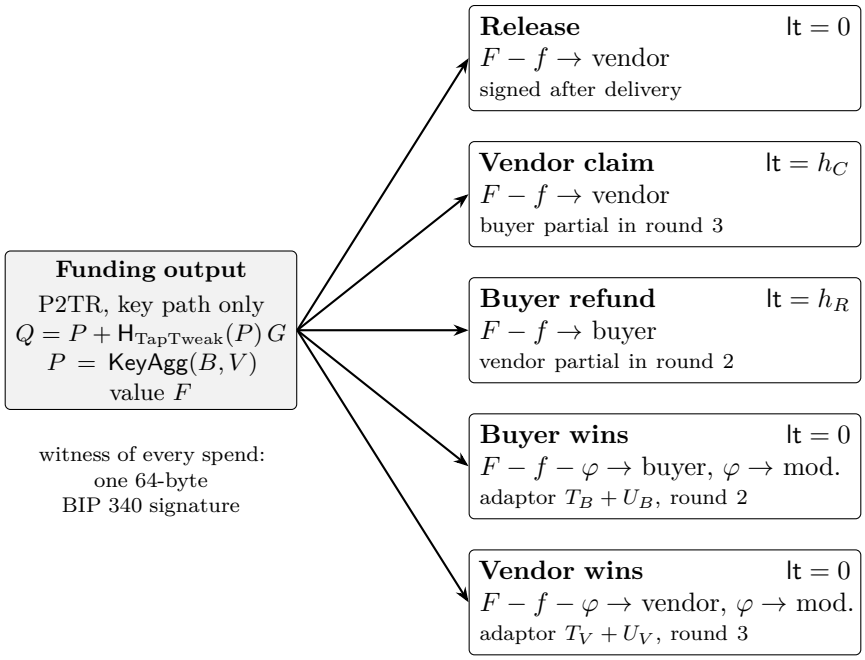


Figure 3: Escrow transaction graph in the two-party protocol (PR #3). All five spends are key-path spends of the same output under the MuSig2 aggregate of the buyer's and vendor's per-order keys; the moderator's key is not part of Q. Four spends are presigned before funding; the release is signed cooperatively later. f is the network fee and φ the moderator fee, both fixed in the signed terms; lt is `nLockTime`.

Spend	`nLockTime`	Outputs	When signed
tx_{rel} (release)	0	$F - f \to S_V$	cooperatively, after delivery
tx_{clm} (vendor claim)	h_C	$F - f \to S_V$	presigned (buyer, round 3)
tx_{ref} (buyer refund)	h_R	$F - f \to S_B$	presigned (vendor, round 2)
tx_{B} (buyer wins)	0	$F - f - \varphi \to S_B$, $\varphi \to S_M$	adaptor Y_B (vendor, round 2)
tx_{V} (vendor wins)	0	$F - f - \varphi \to S_V$, $\varphi \to S_M$	adaptor Y_V (buyer, round 3)

A transaction with `nLockTime` h can first be mined in block $h + 1$.

11.4 Funding rounds

Let N denote a party's MuSig2 public nonces for the four presigned spends and $\mathsf{ps}(\cdot)$ a MuSig2 partial signature [36] (or partial adaptor signature, for tx_{B} and tx_{V}).

Round 0. Both parties sign τ.

Round 1, buyer → vendor. The buyer prepares, but does not sign or broadcast, a funding transaction from native segwit inputs, so its transaction id is fixed before signing. It sends $\big(o,\ \mathsf{N}_B,\ T_B,\ \pi^{\text{pok}}(t_B),\ \mathsf{VEnc}_K(u_V; ctx_V)\big)$. The buyer samples u_V because the buyer loses branch V.

Round 2, vendor → buyer. The vendor checks $\pi^{\text{pok}}(t_B)$ and the encryption proof for U_V (it is the favored party in branch V, so it must be sure the moderator can reveal u_V). It replies $\big(\mathsf{N}_V,\ T_V,\ \pi^{\text{pok}}(t_V),\ \mathsf{VEnc}_K(u_B; ctx_B),\ \mathsf{ps}_V(\mathsf{tx}_{\text{ref}}),\ \mathsf{ps}_V(\mathsf{tx}_{\text{B}}; Y_B)\big)$.

Round 3, buyer → vendor. The buyer checks $\pi^{\text{pok}}(t_V)$ and the encryption proof for U_B, adds its own partial signatures, and verifies that the completed refund is a valid signature under Q for the

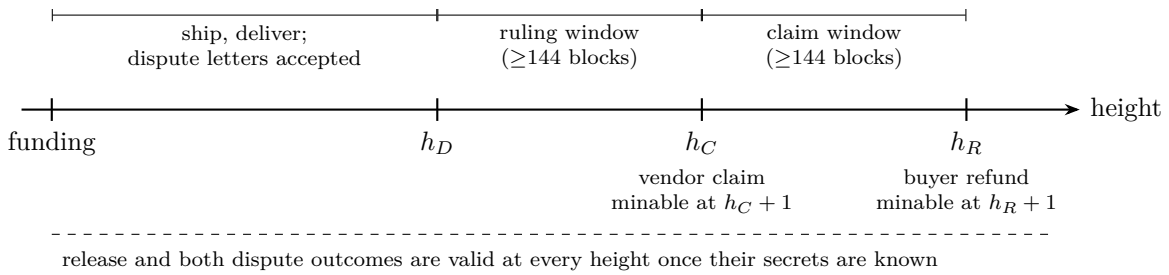


Figure 4: Escrow timeline. The default after h_C is the vendor; the refund at h_R matters only if the vendor never claims. Window sizes are policy minimums checked by each party, 144 blocks each by default.

exact outpoint and value, and that the buyer-wins pre-signature is valid for Y_B. Only after these checks does the buyer's state machine release the funding transaction for signing [implemented, tested]. The buyer must persist the refund before funding; the crate exposes it for storage and re-verification, and storing it is the caller's obligation. The buyer sends $\big(\mathsf{ps}_B(\mathsf{tx}_{\mathrm{clm}}),\ \mathsf{ps}_B(\mathsf{tx}_{\mathrm{V}}; Y_V)\big)$. The vendor completes and stores the claim and verifies the vendor-wins pre-signature.

Funding. The buyer signs and broadcasts the funding transaction. The vendor's state machine checks that a transaction creates exactly the agreed output at o; shipping only after that transaction confirms is the caller's obligation.

Each state is consumed by value when it advances, and secret nonces live only inside those states, so a nonce cannot be reused for two messages by accident. The release uses fresh nonces generated later.

Why each piece is there.

- *Refund before funding.* The buyer never signs the funding transaction until it holds a refund that verifies against the aggregate key. A vendor who aborts after round 1 costs the buyer nothing.
- *Claim before refund.* The buyer presigns a claim at $h_C < h_R$, so a buyer who received the goods and stops responding loses at $h_C + 1$.
- *Proof of knowledge on T_X.* Without it, X could choose $T_X = T' - U_X$ for a T' whose discrete log it knows, making $Y_X = T'$ and letting X complete its own winning branch without the moderator.
- *Verifiable encryption of u_X.* Without it, the losing party $\bar{X}$ could encrypt a value that does not match U_X, and the moderator would be unable to rule for X.
- *Per-order keys.* Tweaking each long-term key with the order secret gives a different Q for every order between the same two parties, so funding outputs between repeat trading partners are not linkable by address [implemented, tested]. The protocol assumes s is shared only by buyer and vendor; carrying it inside the sealed order letter is the intended channel [specified, not implemented].

11.5 Release

When the buyer is satisfied, the vendor sends a fresh public nonce for $\mathsf{tx}_{\mathrm{rel}}$, and the buyer returns its nonce and partial signature. The vendor aggregates them into a 64-byte signature and broadcasts [implemented, tested]. The moderator is not involved.

11.6 Disputes

Before h_D, either party X can ask the moderator to rule for it by sending a *dispute letter* [implemented, tested]:

$$L_X = \big(\tau^*,\ o,\ F,\ \mathit{script}(Q),\ Q,\ X,\ \mathsf{tx_X}\ \text{(unsigned)},\ T_X,\ \mathsf{VEnc}_K(u_X),\ \hat{\sigma}_X,\ \mathit{memo}\big)\ \text{signed by } X.$$

Here $\hat{\sigma}_X$ is the aggregated adaptor pre-signature on $\mathsf{tx_X}$ for Y_X. The letter is self-verifying: the moderator checks the signed terms against its own key, recomputes P, Q, the funding script, and $\mathsf{tx_X}$ from the terms and outpoint, checks that $\hat{\sigma}_X$ is a valid pre-signature on $\mathsf{tx_X}$ for $T_X + U_X$, checks the encryption proof, and checks X's signature over the letter. It refuses a letter that arrives after h_D. Only then does it decrypt u_X.

If it rules for X, the moderator returns a *ruling* (o, X, Y_X, u_X) signed with k. X computes $t_X + u_X$, completes $\hat{\sigma}_X$ into a valid BIP 340 signature, and broadcasts $\mathsf{tx_X}$, which pays X and the moderator's fee.

A ruling for X is useless to $\bar{X}$, because $\bar{X}$ does not know t_X. A moderator that signs rulings for both branches of the same outpoint produces two signed rulings that together are transferable proof of equivocation [implemented, tested]. Equivocation is detectable, not preventable: both dispute outcomes are valid from height 0, so if both parties receive a ruling, the first to confirm wins.

11.7 Timeline

Figure 4 shows the heights.

- From funding until h_D, the vendor ships, the buyer receives, and either party may send a dispute letter.
- Between h_D and h_C, the moderator rules and the winner must confirm its outcome. A buyer who wins must confirm $\mathsf{tx_B}$ before $h_C + 1$, when $\mathsf{tx_{clm}}$ also becomes valid.
- At $h_C + 1$, the vendor can claim. A vendor who never claims leaves the funds in the output until $h_R + 1$, when the buyer's refund becomes valid. If the vendor's claim is not confirmed by then, both are valid and the first to confirm wins.

The default outcome is therefore the vendor's. We considered and rejected a design in which the default flips to the buyer until the vendor posts a shipping attestation: without a trusted third party that observes shipping, such an attestation is not a cryptographic event, and a buyer who wants to cheat would decline to countersign it. The consequence is that a buyer who did not receive the goods must open a dispute before h_D.

11.8 Moderator fee

The moderator fee is an output of each dispute outcome and of nothing else, and it is fixed in the signed terms [implemented, tested]. It is collected only when the winner broadcasts the dispute outcome. A vendor who wins a ruling close to h_C can skip the fee by waiting for the fee-free claim; a buyer who wins cannot. That skews moderator income toward rulings for the buyer. A fee bond posted at funding would remove the skew and is not implemented [future work].

11.9 Verifiable encryption of the moderator half

The moderator half u with $U = uG$ is encrypted bit by bit with exponential ElGamal under K [implemented, tested]. For each bit β_i of u ($i = 0, \ldots, 255$), the encryptor picks r_i and publishes

$(A_i, C_i) = (r_i G,\ r_i K + \beta_i G)$ with a Cramer–Damgård–Schoenmakers OR proof [14] of two Chaum–Pedersen statements [9] showing that $\beta_i \in \{0, 1\}$. A Chaum–Pedersen proof then shows

$$\log_G\Big(\textstyle\sum_i 2^i A_i\Big) = \log_K\Big(\textstyle\sum_i 2^i C_i - U\Big),$$

which holds exactly when $\sum_i 2^i \beta_i \equiv u \pmod{n}$. Anyone with K and U can check the ciphertext; only the holder of k can decrypt, since $C_i - kA_i$ is either the identity or G. All challenges are tagged hashes over length-prefixed inputs and a 32-byte context that binds the proof to one escrow branch. This is the bitwise approach to verifiable encryption of a discrete logarithm [8, 49]. Its cost is size: one encrypted share is 49,761 bytes (§16).

11.10 What the chain shows

Every spend is a single-input Taproot key-path spend with one 64-byte signature. No script, no moderator key, and no multisignature structure appears on chain in any outcome, including a dispute [implemented, tested]. This is the main privacy improvement over the original 2-of-3 design. What remains visible:

- The funding amount F (price plus shipping plus fees), which can point to a listing with a distinctive price.
- Dispute outcomes have two outputs, one of which is the fixed moderator fee φ. If a moderator reuses its payout script S_M, every dispute it rules on is linked on chain to it and to the others. Fresh moderator payout scripts per order are a caller obligation, not enforced.
- The claim and refund carry non-zero `nLockTime` values at specific heights, and all spends signal no replace-by-fee. These are weak fingerprints, since many wallets set `nLockTime` near the current height.
- Timing: a spend shortly after funding versus one after h_C.

11.11 Operational limits

Presigned transactions have fees fixed at funding. If fee rates rise sharply, a presigned spend may not confirm within its window. The winner of any outcome controls an output and could bump the fee by spending that output (child-pays-for-parent); no wallet code does this yet. Each verifiable share is about 50 KB, so a dispute letter is close to the 64 KiB limit of the order mailbox; the transport for escrow messages and dispute letters is not specified in detail. The protocol envelope used for disputes (`DisputeEnvelope` in `crates/protocol`) currently embeds the full order plaintext, including the ship-to field, so a moderator who receives it would learn the address; stripping the ship-to from dispute envelopes is required by the design [specified, not implemented] and not enforced.

12 Discovery and Search

12.1 Search on the device

Search runs over listing headers already held by the client [implemented]: a case-insensitive substring match over title, category, ship-from region, kind, description, and seller name. The query is not sent anywhere, so neither the node, a replica, nor an index learns the search terms. The cost is that the client must hold the headers it searches. We measured this filter at 0.2 ms per query over 1,000 headers and 5 ms over 10,000 headers on a laptop (§16); 100,000 headers took 49–67 ms per query. Matching is literal; there is no ranking, stemming, or index.

12.2 Finding stores and nodes

The intended design [specified, not implemented] treats every source of nodes as untrusted for content and only partly trusted for availability:

- A user can paste a store link (a store key plus an onion address). A store reached this way opens with an empty seed list.
- Seeds shipped with the software, and peers remembered from earlier sessions, are used to find replicas.
- Every record received from any of them is verified against its signature and content hash before use.

Because records are self-authenticating, a malicious seed or replica cannot forge or alter a listing. It can withhold records, serve stale ones (§7), or, if it controls all of a node's peers, present a partial view of the network: an eclipse attack [26]. Remembering peers across sessions, drawing peers from several independent sources, and preferring a pasted link over seed-supplied data are the intended mitigations; none is implemented, and no Sybil defense [18] is designed beyond the publishing cost of §7.

12.3 Private search at scale

On-device search stops scaling when the catalog no longer fits on a phone. The next step in the design is private information retrieval (PIR) [10] against a periodic snapshot of the header catalog, so that a replica answers a query without learning which records were fetched. Single-server lattice-based schemes such as SimplePIR [27] make this plausible for databases of the relevant size. Nothing here is implemented or designed in detail [future work]. Until then, a client that fetches individual records from a replica reveals its interest in those records to that replica.

13 Security Analysis

We state each property with its assumptions and its implementation status. The arguments are informal; a machine-checked or game-based proof is future work. The escrow properties (P1–P4) refer to the two-party protocol in PR #3 and hold only for code that uses it; the merged single-process escrow on main and in PR #2 does not satisfy P1, because one process holds both secret keys.

Assumption 1 (Escrow)**.** (a) MuSig2 with BIP 327 key aggregation is unforgeable [36], and adaptor pre-signatures do not let anyone without the adaptor secret produce a valid signature. (b) The proofs of knowledge, the bit proofs, and the DLEQ proof are sound in the random-oracle model. (c) Each party runs the protocol checks in its own state machine before advancing and keeps its own secret key. (d) The funding transaction spends only native segwit inputs, so its transaction id cannot be changed after the presigned spends are created. (e) A party who needs to broadcast a transaction gets it confirmed within the relevant window (at least 144 blocks by default) at the presigned fee rate or with a fee bump.

Property 1 (No custody)**.** *No party other than the buyer and the vendor holds a key sufficient to spend the escrow output.*

Argument. The output key Q is a tweak of $\mathsf{KeyAgg}(B, V)$; the moderator's key K is not aggregated. Under Assumption 1(a), producing any valid signature under Q requires partial signatures from

both b and v. Every spend that can exist was either presigned by both parties before funding or is signed by both cooperatively (the release). *Status:* [implemented, tested] in the escrow crate of PR #3; not yet used by the node.

Property 2 (Bounded moderator)**.** *A moderator, alone or with any outside party, cannot move the funds, and cannot cause any payout other than one of the two dispute outcomes whose outputs both parties signed.*

Argument. The moderator learns u_X for at most the branch it rules on. A completed signature on tx_X needs $t_X + u_X$ and the pre-signature $\hat{\sigma}_X$. The moderator does not know t_X, and by the proof of knowledge (Assumption 1(b)) X could not have chosen T_X to cancel U_X. The outputs of tx_X, including the moderator fee and payout scripts, are fixed by the signed terms and committed in its signature hash, so the moderator cannot redirect funds or change amounts. A moderator not named in the signed terms is rejected by both parties before any signature is produced. *Status:* [implemented, tested] in PR #3. The property concerns the buyer's and vendor's payouts; whether the moderator fee output is standard and actually reaches the moderator is not checked at `dc7bc42` (finding H2).

What the moderator can do: rule for the wrong party; refuse to rule, which leaves the parties to the claim and refund; or rule for both parties, which lets both broadcast and makes the outcome a race. The last is provable from two signed rulings but not preventable. A moderator colluding with one party (A5) can make that party win the dispute, which is the same economic trust as in a 2-of-3 escrow.

Property 3 (Buyer exit)**.** *If the vendor disappears after funding, the buyer recovers $F - f$ at height $h_R + 1$ without anyone's cooperation. If the vendor stays but does not deliver, the buyer recovers $F - f - \varphi$ provided it sends a dispute letter by h_D, the moderator rules for it, and the outcome confirms by h_C.*

Argument. The buyer holds a complete refund signature before it signs the funding transaction (refund before funding), and the refund needs nothing but height $h_R + 1$. If the vendor never broadcasts the claim, nothing else can spend the output before then except the release (which the buyer must sign) and the dispute outcomes (which need a ruling). For the second case, the buyer holds the pre-signature for tx_B and the moderator can reveal u_B because the buyer verified its encryption in round 3. *Limits:* the buyer cannot always exit. Against a vendor who stays online and does not deliver, the buyer depends on the moderator ruling in time; if the moderator does not, the vendor claims at $h_C + 1$. *Status:* [implemented, tested] in PR #3 (tests cover refund-before-funding, a buyer who disputes in time, and letters refused after h_D). At `dc7bc42` this property additionally requires the buyer to check that h_D lies far enough above the current chain tip, because the code does not (finding H1, §13.2).

Property 4 (Vendor protection against a stalling buyer)**.** *If the buyer stops responding after funding, the vendor collects $F - f$ at height $h_C + 1$ without anyone's cooperation.*

Argument. The vendor holds a complete claim signature from round 3 with `nLockTime` $h_C < h_R$. The buyer's only competing unilateral spend is the refund, which is not valid until $h_R + 1$, and a buyer dispute needs a ruling. *Assumption:* the vendor broadcasts and confirms the claim inside the claim window (Assumption 1(e)). *Status:* [implemented, tested] in PR #3 (test: a stalling buyer loses at the vendor claim height). The same mechanism is what H1 abuses: with heights already in the past, the "stalling buyer" default fires immediately after funding.

Property 5 (Catalog integrity)**.** *A party that does not hold a store's secret key cannot produce a store record or listing that verifies under that store's key, and cannot alter a listing without changing its address.*

Argument. BIP 340 signatures are existentially unforgeable under the discrete-log assumption in the random-oracle model. The tagged hash separates record types. The canonical encoding is injective and rejects trailing bytes, so two different records cannot share a signed body. The listing address is SHA-256 of the body, so changing any field changes the address under collision resistance. *Limits:* no freshness and no completeness (§7). *Status:* [implemented, tested] on main.

Property 6 (Letter confidentiality)**.** *Only the holder of the vendor's mailbox secret can read an order letter, and only the holder of the buyer's per-order reply secret can read a reply.*

Argument. HPKE base mode is IND-CCA2 secure for the recipient under the Gap Diffie–Hellman assumption on X25519 and AEAD security of ChaCha20-Poly1305 [4]. Distinct `info` strings separate orders from replies. *Assumption:* the buyer seals to the correct key. Today the key comes from a compiled-in fixture table, not from a signed store record (§8); until the binding exists, anyone who can change the key a buyer looks up can read that buyer's letters. *Status:* sealing [implemented, tested]; key binding [specified, not implemented].

13.1 Other attacks considered

Rogue keys. BIP 327 key aggregation weights each key with a coefficient that depends on all keys, which prevents a party from choosing its key to cancel the other's [34]. The party halves T_X are protected by proofs of knowledge, as above.

Nonce reuse. MuSig2 secret nonces live inside party states that are consumed when they advance and cannot be cloned; the release draws new nonces, distinct from those of the dispute outcome that pays the same party the same amount.

Garbage moderator share. A losing party that encrypts a value not matching U_X is caught by the favored party's check before that party signs anything leading to funding (tests cover both directions).

Late dispute letters. The moderator refuses letters received after h_D, so a ruling always has the full ruling window before the vendor claim becomes valid. The moderator's notion of "received" is its own view of the chain height.

Client impersonation on the same machine. Any local process can obtain the pairing token (§6) and place orders or read the address book. This is in scope for remote attackers and other browser origins, and out of scope for local processes.

13.2 Adversarial review of the two-party escrow

The PR #3 escrow was reviewed adversarially by a reviewer who did not write it. The review found the cryptographic core sound: the bitwise ElGamal verifiable encryption with per-bit OR-proofs and a DLEQ link is the standard bit-decomposition approach [49]; no rogue-key or cancellation attack was found; no party could obtain a signature outside the presigned set; nonces were fresh and single-use; funding was withheld until the refund and the buyer-wins pre-signature verified; and

20,000 fuzzed decoder inputs produced no panics. The findings were in the policy checks around that core. Table 3 lists them with their status on `escrow-two-party` at `dc7bc42`. "Open" means no fix was on the branch at that commit; "fixed" means a fix and a test for it were on the branch; "proposal only" means the fix is a design that is not implemented.

Table 3: Findings of the escrow review and their status at `dc7bc42`.

ID	Finding	Consequence and planned fix	Status
H1	Terms whose heights are already below the current chain tip are accepted; nothing compares h_D to the tip.	A vendor can propose past heights and claim right after funding (reviewer's example: heights 1000/1144/1288 with the claim broadcast at height 900,001). Fix: both sessions take the current tip; require h_D at least a delivery window above it and cap h_R.	open
H2	The moderator payout script may be nonstandard, below its type's dust limit, or the vendor's own script.	Silently disables or redirects the moderator fee. Fix: allow only standard output types with per-type dust; compare script and fee with the moderator's published values on all three sides.	open
H3	The node still uses the single-process legacy escrow.	The escrow anyone can run is the unsafe demonstration (mitigated in PR #2 by refusing escrow on mainnet). Fix: move the node to the two-party API.	open
M1	Rulings sign the outpoint, the favored party, and the adaptor point, but not a digest of the terms.	Equivocation proofs could be framed with rulings from different terms. Fix: sign the terms digest and output key; only count rulings over the same digest; give the moderator a chain check.	open
M2	The moderator API is stateless and can release both branches.	Accidental equivocation. Fix: a persistent ruling store keyed by terms digest and outpoint; refuse the second branch.	open
M3	A vendor who wins near h_C can wait for the fee-free claim.	The moderator is in practice paid only for buyer wins, a bias toward the buyer. Fix: a fee bond at funding, or a moderator fee in the claim.	proposal only
M4	Fixed presigned fees and no anchor output.	A fee race near h_C. Fix: pay-to-anchor outputs with version-3 (TRUC) transactions so the recipient can bump by CPFP, or a higher fee floor.	open
M5	The moderator's local policy is private and could reject every letter.	Disputes silently impossible. Fix: publish the policy, put it in the signed terms, check it at funding.	open
M6	Each share is about 50 KB, and the review measured 0.2–0.27 s per verification.	A denial-of-service lever against moderators and counterparties. Fix: remove a duplicate verification, cache verified letters, batch the bit-proof checks.	open
L	Escrow states are `Clone`; no maximum network fee; the release cannot be resigned before broadcast; funding replacement is not forbidden; the moderator uses one key for signing and decryption.	Low severity. Fix as listed.	open

Our own release-build measurement of one share verification is 52 ms (§16); we did not reconcile it with the review's figure, which may count a full letter check or a different build. H1, H2, M1, M2, and M5 were classed as must-fix. We treat all open findings as blocking for any mainnet use of the two-party escrow, and adopt the review's recommendation that the protocol receive an external cryptographic review before then. The review also noted smaller alternatives to the 50 KB share, such as segmented ElGamal with range proofs as in JugglingSwap [47], and that Camenisch–Shoup verifiable encryption [8] fits poorly because it needs a Paillier-style group.

14 Privacy Analysis

Table 4 lists, for each party or observer, what it learns in the intended design and which parts of that depend on unimplemented mechanisms. "Today" describes the code as it runs; in today's demonstration buyer and vendor share one node and no traffic leaves the machine except chain queries (PR #2) and exchange-rate requests.

Table 4: Who learns what.

Observer	Learns	Does not learn	Depends on
Seller	Items, quantities, and options in its own package; the ship-to for physical goods or the delivery note for digital goods; the payment and its amount; with Payjoin, the buyer's inputs.	The buyer's IP address; the buyer's orders with other sellers; any long-term buyer identity (none is sent).	IP hiding needs Tor delivery [specified, not implemented]; separation from other sellers needs coin selection that avoids shared change [future work].
Carrier	Ship-to, parcel, and delivery timing.	Price paid, payment method, other orders.	This is the ceiling for physical goods.
Moderator, no dispute	Nothing about the order. It is named in public listings.	That a given order used it; its key is not on chain.	[implemented, tested] in PR #3.
Moderator, dispute	Signed terms (per-order keys, amounts, heights, payout scripts), the outpoint and therefore the on-chain funding and spends, the claimant's memo. Under today's dispute envelope, also the order plaintext including ship-to.	Other orders of either party.	Stripping ship-to from dispute envelopes [specified, not implemented].
Replica	Which records a node fetches or publishes, and when; with clearnet transport, from which IP.	Orders; search terms (search is on the device).	Replication [specified, not implemented]; Tor [specified, not implemented]; PIR [future work].
Seed	That a node speaks the protocol, when, and which peers it asks for; IP without Tor.	Records fetched elsewhere; orders.	Seeds [specified, not implemented].

Observer	Learns	Does not learn	Depends on
Chain observer	Amounts and timing of every payment; for escrow, a single-key-looking Taproot output and spend; a two-output dispute outcome with a fixed fee; links through shared inputs or change. Prices are public, so a distinctive amount can identify a listing.	Moderator keys, escrow scripts, that a dispute occurred (beyond output pattern), ship-to.	Key-path escrow [implemented, tested] in PR #3; Payjoin [future work].
Esplora server	Every address the node watches, with timing and the node's IP; this links all of a user's orders together.	Order contents.	Today: clearnet, `mempool.space` default. Tor [specified, not implemented]; own node [future work].
Tor relays	Guard: the node's IP and that it uses Tor. Rendezvous and middle relays: neither endpoint. Approximate message sizes.	Letter contents; the vendor's IP; with one relay, who talks to whom.	Tor delivery [specified, not implemented].
Lightning hops	Forwarded amounts, timing, and adjacent channels. Without a blinded path, the payer learns the recipient's node key.	Order contents.	Lightning payment [future work] (offers are encoding only).
Exchange-rate endpoint	The node's IP and its polling times (at most once per five minutes when the shop asks).	What was bought.	[implemented, tested]; Coinbase by default; no Tor.
Local network or ISP	Today: TLS connections to the Esplora server and the exchange-rate endpoint. With Tor: that the user runs Tor.	Client–node traffic, which stays on loopback.	[implemented, tested] loopback.
Global passive adversary	Can correlate a buyer node's Tor circuit timing with a vendor onion service, and both with chain timing, linking buyer to vendor.	Letter contents.	Out of scope; mixnet for letters [future work].
Same-machine process	The pairing token, then everything the client can see.	—	Out of scope.

The ship-to ceiling. For physical goods, the seller and the carrier learn where the buyer receives parcels. No protocol mechanism removes this, and we do not claim to. What the design does is keep the ship-to out of every other place: it is not in the catalog, not in any public record, not in the buyer's own order history on disk (which stores a label), and, once delivery runs over Tor, not exposed to network observers. Tests check that the ship-to does not appear in the catalog response or in the mailbox file in plaintext.

Linking a buyer's orders. Three channels can link one buyer's orders across sellers today: the Esplora server, which sees all watched addresses from one IP; the chain, if payments share inputs or change; and the exchange-rate endpoint, which sees the node's IP but not orders. Separate letters and separate transactions are necessary for unlinkability, and these remaining channels mean they are not yet sufficient.

15 Residual Trust and Limitations

We collect the limitations stated elsewhere, ordered roughly by how much they matter to a user today.

1. **No network.** Nothing travels over Tor. In the demonstration, buyer and vendor share one node and the letter is written to that node's own mailbox. Every network-privacy statement in this paper about Tor is a design statement.
2. **Escrow is not two-party in any running node.** The node uses the merged single-process escrow, in which one process holds both secret keys. It is restricted to testnet4 in PR #2 and is labeled unsafe. The two-party protocol exists as a tested library in PR #3 with no transport.
3. **The moderator is trusted to rule, and to rule in time.** A moderator that does not rule leaves a non-delivered buyer to lose at $h_C + 1$. A moderator colluding with one party decides for that party. Equivocation is provable, not preventable.
4. **The buyer must act before h_D.** The default outcome after h_C is the vendor's, so a buyer who does not dispute non-delivery in time loses the payment.
5. **Chain queries leak.** The Esplora server, and on testnet4 its fallback once used, sees every watched address and the node's IP address.
6. **The mailbox key is not bound to the store.** Until the signed store record names the mailbox key, a buyer's letter is only as confidential as the lookup that told it which key to use.
7. **At-rest encryption keeps its key beside the data.** It is not a defense against anyone who can read the data directory.
8. **Local processes can act as the client.** The pairing token is available to any process on the machine.
9. **No catalog freshness, completeness, replication, or spam cost.** A replica can serve stale listings or omit them; a store is available only from its own node today.
10. **Payment rails beyond on-chain are not live.** BOLT 12 offers and BIP 352 codes are valid strings that nothing answers or scans; Payjoin is a library.
11. **Amounts can identify listings, and payments can be linked.** Public prices let a chain observer guess what was bought from the payment amount, and because the wallet spends its own unconfirmed change, consecutive payments to different sellers can be chained together on chain.
12. **Presigned fees.** Presigned escrow spends have fixed fees and no anchor output, so fee bumping is not possible and a fee spike near h_C can let the vendor's claim race the buyer's dispute (finding M4).
13. **No padding.** Letter sizes leak approximate content length.
14. **Open review findings; no external review.** An internal adversarial review of the two-party escrow found the issues in Table 3; at `dc7bc42` their status is as listed there. The code has not been audited by a third party, and the security arguments here are informal.

16 Implementation and Evaluation

16.1 Code

Table 5 gives the size of each crate in lines of Rust, including tests, at each commit we cite. The client in `apps/mobile` is about 2,400 lines of TypeScript.

Crate	main `b015bff`	PR #2 `58e539e`	PR #3 `dc7bc42`
`protocol`	1,475	1,475	1,534
`tormail`	1,498	1,498	1,498
`pay`	1,051	1,092	1,051
`escrow`	1,760	1,760	4,472
`node`	3,030	5,853	3,030

Table 5: Lines of Rust per crate, including unit tests.

16.2 Tests

We ran `cargo test` on each commit (Table 6). Two tests are ignored by default because they need the live network: one bootstraps Arti and dials an onion address, and one (PR #2) pays real testnet4 coins. We did not run either. The PR #3 count is for the escrow and protocol crates, built in release mode; the node at that commit is unchanged from main.

Commit	`node`	`escrow`	`pay`	`protocol`	`tormail`	Total
main `b015bff`	17	6	11	12	9 (+1 ign.)	55 passed, 1 ignored
PR #2 `58e539e`	40 (+1 ign.)	6	13	12	9 (+1 ign.)	80 passed, 2 ignored
PR #3 `dc7bc42`	—	26 + 3 doc	—	12	—	41 passed

Table 6: Automated tests we ran. All passed.

The tests that matter most for the claims in this paper are, by name: `seals_the_ship_to_into_the_mailbox_and_keeps_it_out_of_the_catalog`, `multi_seller_checkout_seals_one_letter_per_vendor`, `rejects_a_client_supplied_destination`, `pairing_token_is_stable_and_orders_require_it`, `address_book_is_encrypted_and_stays_off_the_catalog`, and `wrong_key_cannot_open` on main; and, in PR #3, `two_parties_fund_by_exchanging_only_messages`, `each_party_session_takes_only_its_own_key`, `funding_psbt_is_only_released_after_a_verified_refund`, `stalling_buyer_loses_at_the_vendor_claim_height`, `buyer_who_disputes_in_time_wins`, `moderator_share_alone_does_not_finish_a_branch`, `rulings_are_signed_and_equivocation_is_provable`, `per_order_keys_make_orders_between_the_same_pair_unlinkable`, and `buyer_rejects_a_garbage_buyer_wins_share_and_never_sees_the_psbt`.

16.3 Measurements

We measured the operations whose cost determines whether the design is practical on ordinary hardware. All numbers are from one machine (Apple M2 Max laptop, 32 GB, macOS 26.7, Rust 1.94 release builds, Node.js 24.6), averaged over the stated number of repetitions after a warm-up. They are microbenchmarks of local computation; we did not measure anything over a network, Tor, or a phone. Table 7 lists them.

Operation	Result	Repetitions
Encoded listing (fixture paperback)	268 bytes	—
Encoded fixture catalog (1 store, 38 listings)	9,940 bytes	—
Sign one listing (BIP 340)	101 μs	2,000
Verify one listing	48 μs	2,000
Verify the whole fixture catalog	1.9 ms	200
Order plaintext with a four-line US ship-to	173 bytes	—
Sealed letter for that order	230 bytes	—
HPKE seal one order	115 μs	2,000
HPKE open one order	75 μs	2,000
Verifiable moderator share, size (by the serialization format)	49,761 bytes	—
Encrypt one share	44 ms	20
Verify one share	52 ms	20
Decrypt one share	54–57 ms	20
On-device search, 1,000 headers	0.2 ms/query	100
On-device search, 10,000 headers	5 ms/query	100
On-device search, 100,000 headers	49–67 ms/query	100, two runs

Table 7: Measured costs. Search headers were synthesized by repeating the fixture headers with distinct titles; queries were a mix of matching and non-matching terms.

The share size follows from the format: one 33-byte point, 256 ciphertext pairs of two 33-byte points each, 256 OR-proofs of four 32-byte scalars each, and a 64-byte DLEQ proof: $33 + 256 \cdot (66 + 128) + 64 = 49{,}761$. An escrow funding exchange carries two shares (one per dispute branch), so about 100 KB, and a dispute letter carries one. Share decryption tests each of 256 bits and is about as costly as verification.

These numbers support three practical conclusions. Signing and verifying the catalog and sealing orders cost well under a millisecond each, so they are not a bottleneck. On-device search over tens of thousands of headers is interactive on a laptop; we have not measured it on a phone. The verifiable share dominates the escrow's cost in both bytes and time, which is why finding M6 and the smaller-proof alternatives in §13.2 matter.

17 Related Work

Peer-to-peer marketplaces. The original OpenBazaar [40] used IPFS-style content addressing [5] for stores, Ricardian-style signed contracts [24] for orders, and 2-of-3 multisignature escrow with a moderator. This design keeps the signed-contract idea and replaces the rest: catalogs are signed records rather than a node's public content, orders are sealed letters rather than messages between online peers, and escrow is a two-party key with an encrypted moderator share rather than a moderator-held key. Particl [42] also runs a decentralized marketplace and uses two-party deposits in which both sides lock funds, which removes the moderator but requires the buyer and vendor to over-collateralize. Bisq [6] and Haveno [25] are peer-to-peer exchanges for currency trades rather than goods; they use 2-of-2 or 2-of-3 deposits with arbitrators and security deposits, and their arbitration roles inform our moderator design. Nostr's NIP-15 [39] defines marketplace stalls and products as signed events on relays [22]; it shares our signed-catalog model but carries orders as encrypted direct messages and has no escrow.

Earlier anonymous markets. Measurement studies of Silk Road and its successors [12, 48] document how centralized escrow, operator custody of funds, and server seizure shaped those markets. We cite them for the custody and single-operator lessons; OpenBazaar's scope is ordinary legal goods, and its privacy mechanisms are those any retail buyer has reason to want.

Payments. Lightning [44] and BOLT 12 offers [45] give fast payments whose privacy depends on routing and blinded paths; Kappos et al. [30] measure how much Lightning payments leak in practice. Silent payments (BIP 352) [3] let a payer derive a fresh output for a static public code; payjoin (BIP 78, BIP 77) [17, 23] breaks the common-input heuristic [35]. All three appear in this design as encoding or library code only (§10).

Escrow and signatures. The two-party key uses MuSig2 [36, 37], a successor to MuSig [34], on BIP 340/341 outputs [54, 55]. Adaptor signatures [1] let a completed signature reveal a secret, which is how the moderator's share finishes one branch. Discreet Log Contracts [19] use an oracle's signature as an adaptor secret to select among pre-signed outcomes; our moderator plays a similar role but is told the dispute only if one is raised and learns its secret through verifiable encryption rather than publishing an attestation. The verifiable encryption follows Stadler's publicly verifiable secret sharing [49], built from exponential ElGamal [20], Chaum–Pedersen equality proofs [9], and CDS OR-composition [14] under Fiat–Shamir [21]. Camenisch–Shoup verifiable encryption [8] gives much smaller proofs for discrete logarithms but needs a different group setting, and JugglingSwap [47] gives a segmented variant closer to ours. A threshold moderator panel could use FROST [13, 31].

Anonymous transport and private retrieval. Tor onion services [16, 51] hide the server's location; Johnson et al. [29] quantify what a realistic adversary learns about Tor users over time. Mix networks such as Loopix [43] and Nym [15] add cover traffic and delay to resist a global passive adversary, at a latency cost an order mailbox can afford. Oblivious HTTP [52] separates who asks from what is asked for a single request. Private information retrieval [10], now practical at the scale of a catalog index with SimplePIR [27], is the natural next step for discovery (§12). The replication design must also contend with eclipse [26] and Sybil [18] attacks, for which proof-of-work stamps [2] are one partial answer.

18 Conclusion

We have described a marketplace in which the public catalog contains only what a seller chooses to publish, a buyer's shipping address is sealed to one seller and never enters the catalog, payments are non-custodial, and escrow is a two-party key in which a moderator can decide a dispute without ever being able to take the funds. Each mechanism carries a label. Signed catalogs, sealed orders, and the per-order wallet are implemented and tested. The two-party escrow is implemented and tested on an open pull request, but an internal review found open issues that block its use with real money. Tor delivery, a mailbox key bound to the store record, replica and seed networks, and private discovery are specified or future work.

The limits are as important as the mechanisms. The seller and the carrier must learn the ship-to. The chain records every payment. A network observer who can watch both ends can correlate timing. A moderator in a dispute learns the order. What the design changes is who else learns these things, and that no party can hold the buyer's or seller's funds alone. Before mainnet, the work that matters most is to fix the open review findings, deliver orders over Tor to a key bound to

the store record, replace fixture keys with keys from the wallet seed, and obtain an external review of the escrow protocol.

References

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *Advances in Cryptology – ASIACRYPT 2021*, volume 13091 of *LNCS*, pages 635–664. Springer, 2021.

[2] Adam Back. Hashcash – a denial of service counter-measure. `http://www.hashcash.org/papers/hashcash.pdf`, 2002.

[3] Josie Baker, Ruben Somsen, and Sebastian Falbesoner. BIP 352: Silent payments. Bitcoin Improvement Proposal, 2023.

[4] Richard Barnes, Karthikeyan Bhargavan, Benjamin Lipp, and Christopher A. Wood. Hybrid public key encryption. RFC 9180, 2022.

[5] Juan Benet. IPFS – content addressed, versioned, P2P file system. arXiv:1407.3561, 2014.

[6] Bisq contributors. Bisq: A decentralized bitcoin exchange network. `https://bisq.network`, source at `https://github.com/bisq-network/bisq`.

[7] Blockstream. Esplora: Block explorer and HTTP API. `https://github.com/Blockstream/esplora`.

[8] Jan Camenisch and Victor Shoup. Practical verifiable encryption and decryption of discrete logarithms. In *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *LNCS*, pages 126–144. Springer, 2003.

[9] David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Advances in Cryptology – CRYPTO '92*, volume 740 of *LNCS*, pages 89–105. Springer, 1992.

[10] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 41–50, 1995.

[11] Ava Chow. BIP 86: Key derivation for single key P2TR outputs. Bitcoin Improvement Proposal, 2021.

[12] Nicolas Christin. Traveling the Silk Road: A measurement analysis of a large anonymous online marketplace. In *Proceedings of the 22nd International Conference on World Wide Web (WWW)*, pages 213–224, 2013.

[13] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. The flexible round-optimized Schnorr threshold (FROST) protocol for two-round Schnorr signatures. RFC 9591, 2024.

[14] Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *Advances in Cryptology – CRYPTO '94*, volume 839 of *LNCS*, pages 174–187. Springer, 1994.

[15] Claudia Diaz, Harry Halpin, and Aggelos Kiayias. The Nym network: The next generation of privacy infrastructure. Nym Technologies white paper, 2021.

[16] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. In *Proceedings of the 13th USENIX Security Symposium*, pages 303–320, 2004.

[17] Nicolas Dorier. BIP 78: A simple payjoin proposal. Bitcoin Improvement Proposal, 2019.

[18] John R. Douceur. The Sybil attack. In *Peer-to-Peer Systems: First International Workshop (IPTPS 2002)*, volume 2429 of *LNCS*, pages 251–260. Springer, 2002.

[19] Thaddeus Dryja. Discreet log contracts. `https://adiabat.github.io/dlc.pdf`, 2017.

[20] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.

[21] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO '86*, volume 263 of *LNCS*, pages 186–194. Springer, 1986.

[22] fiatjaf and Nostr contributors. NIP-01: Basic protocol flow description. Nostr Implementation Possibilities, `https://github.com/nostr-protocol/nips`.

[23] Dan Gould and Yuval Kogman. BIP 77: Async payjoin. Bitcoin Improvement Proposal, 2023.

[24] Ian Grigg. The Ricardian contract. In *Proceedings of the First IEEE International Workshop on Electronic Contracting*, pages 25–31, 2004.

[25] Haveno contributors. Haveno: A decentralized Monero exchange. `https://github.com/haveno-dex/haveno`.

[26] Ethan Heilman, Alison Kendler, Aviv Zohar, and Sharon Goldberg. Eclipse attacks on Bitcoin's peer-to-peer network. In *Proceedings of the 24th USENIX Security Symposium*, pages 129–144, 2015.

[27] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *Proceedings of the 32nd USENIX Security Symposium*, 2023.

[28] Fabian Jahr. BIP 94: Testnet 4. Bitcoin Improvement Proposal, 2024.

[29] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr, and Paul Syverson. Users get routed: Traffic correlation on Tor by realistic adversaries. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 337–348, 2013.

[30] George Kappos, Haaroon Yousaf, Ania M. Piotrowska, Sanket Kanjalkar, Sergi Delgado-Segura, Andrew Miller, and Sarah Meiklejohn. An empirical analysis of privacy in the Lightning network. In *Financial Cryptography and Data Security (FC 2021)*, volume 12674 of *LNCS*, pages 167–186. Springer, 2021.

[31] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *Selected Areas in Cryptography – SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, 2020.

[32] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, 2010.

[33] Adam Langley, Mike Hamburg, and Sean Turner. Elliptic curves for security. RFC 7748, 2016.

[34] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple Schnorr multi-signatures with applications to Bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, 2019.

[35] Sarah Meiklejohn, Marjori Pomarole, Grant Jordan, Kirill Levchenko, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. A fistful of bitcoins: Characterizing payments among men with no names. In *Proceedings of the 2013 Internet Measurement Conference (IMC)*, pages 127–140, 2013.

[36] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In *Advances in Cryptology – CRYPTO 2021*, volume 12825 of *LNCS*, pages 189–221. Springer, 2021.

[37] Jonas Nick, Tim Ruffing, and Elliott Jin. BIP 327: MuSig2 for BIP340-compatible multi-signatures. Bitcoin Improvement Proposal, 2022.

[38] Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, 2018.

[39] Nostr contributors. NIP-15: Nostr marketplace. Nostr Implementation Possibilities, `https://github.com/nostr-protocol/nips`.

[40] OpenBazaar contributors. openbazaar-go: OpenBazaar 2.0 server daemon in Go. `https://github.com/OpenBazaar/openbazaar-go`. MIT license; last commit on master 20 November 2020.

[41] Marek Palatinus, Pavol Rusnak, Aaron Voisine, and Sean Bowe. BIP 39: Mnemonic code for generating deterministic keys. Bitcoin Improvement Proposal, 2013.

[42] Particl contributors. Particl marketplace. `https://particl.io`, source at `https://github.com/particl`.

[43] Ania M. Piotrowska, Jamie Hayes, Tariq Elahi, Sebastian Meiser, and George Danezis. The Loopix anonymity system. In *Proceedings of the 26th USENIX Security Symposium*, pages 1199–1216, 2017.

[44] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning network: Scalable off-chain instant payments. `https://lightning.network/lightning-network-paper.pdf`, 2016.

[45] Rusty Russell. BOLT #12: Negotiation protocol for lightning payments. Lightning Network specifications, `https://github.com/lightning/bolts`, 2024.

[46] Nils Schneider and Matt Corallo. BIP 21: URI scheme. Bitcoin Improvement Proposal, 2012.

[47] Omer Shlomovits and Oded Leiba. JugglingSwap: Scriptless atomic cross-chain swaps. arXiv:2007.14423, 2020.

[48] Kyle Soska and Nicolas Christin. Measuring the longitudinal evolution of the online anonymous marketplace ecosystem. In *Proceedings of the 24th USENIX Security Symposium*, pages 33–48, 2015.

[49] Markus Stadler. Publicly verifiable secret sharing. In *Advances in Cryptology – EUROCRYPT '96*, volume 1070 of *LNCS*, pages 190–199. Springer, 1996.

[50] The Tor Project. Arti: An implementation of Tor in Rust. `https://gitlab.torproject.org/tpo/core/arti`, .

[51] The Tor Project. Tor rendezvous specification – version 3. `https://spec.torproject.org/rend-spec/`, . Accessed November 2026.

[52] Martin Thomson and Christopher A. Wood. Oblivious HTTP. RFC 9458, 2024.

[53] Pieter Wuille. BIP 32: Hierarchical deterministic wallets. Bitcoin Improvement Proposal, 2012.

[54] Pieter Wuille, Jonas Nick, and Tim Ruffing. BIP 340: Schnorr signatures for secp256k1. Bitcoin Improvement Proposal, 2020.

[55] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 341: Taproot: SegWit version 1 spending rules. Bitcoin Improvement Proposal, 2020.

A Wire Formats

All integers are little-endian. `str` is a 2-byte length followed by UTF-8 bytes. Every format begins with a 4-byte magic and a 1-byte version, and decoders reject unknown versions and trailing bytes. The escrow crate's signed-terms wrapper reuses the magic `OBST`; it is never parsed in the same context as a store record, but a future version should give it a distinct magic.

Magic	Record	Fields after magic and version
`OBST` v1	Store	name (str), region (str), BOLT 12 offer (str), BIP 352 code (str), store key (32 bytes), then a 64-byte BIP 340 signature over the preceding bytes with tag `ob/store/v1`.
`OBLS` v4	Listing	store key (32), title (str), category (str), price (u64), ship-from (str), description (str), seller (str), stock (u32), listed-on date (u32), moderated (u8), kind (u8), product (str), shipping (u64), free-over (u64), then a signature with tag `ob/listing/v1`. Address: SHA-256 of everything before the signature.
`OBOD` v1	Order plaintext	listing address (32), quantity (u32), option count (u16) and name/value pairs (str, str), ship-to (str), rail (u8), reply key (32).
`OBSL` v1	Sealed letter	HPKE encapsulated key (32), ciphertext length (u32), ciphertext. At most 64 KiB.
`OBSR` v1	Sealed reply	As `OBSL`, sealed to the order's reply key; at most 8 KiB.
`OBML` v1	Onion frame	Letter length (u32), then one `OBSL` letter.
`OBET` v1	Escrow terms	The keys, amounts, heights, and payout scripts of §11.2.
`OBDL` v2	Dispute letter	The fields of the dispute letter in §11, then the claimant's signature.
`OBRU` v1	Ruling	Funding outpoint, favored party (u8), adaptor point (33), moderator share (32), moderator signature (64).

B Reproducing the Measurements

The test counts in §16 come from `cargo test --workspace` at main `b015bff` and PR #2 `58e539e`, and from `cargo test --release -p openbazaar-escrow -p openbazaar-protocol` at PR #3 `dc7bc42`.

The timing programs call only public functions: `Listing::sign`, `Listing::verify`, `Catalog::verify`, `seal_order`, and `open_order` for the catalog and sealing rows; `VerifiableShare::encrypt`, `verify`, and `open` for the escrow rows; and the client's `filterListings` for search. They are not committed with the paper.